📘 CHATBOT_DISTILBERT – Complete Project Documentation

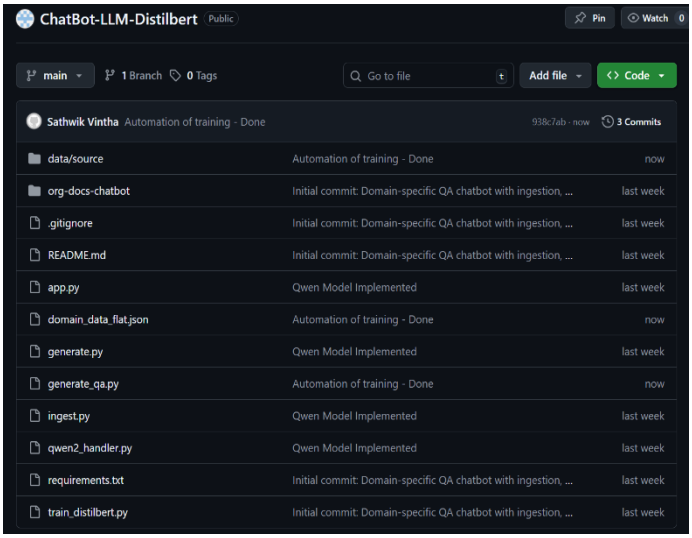
1. Project Overview

CHATBOT_DISTILBERT is an **AI-powered document question-answering system**.
It is an end-to-end document-based Question Answering (QA) system. It allows users to upload documents (PDF, DOCX, TXT) and later ask natural-language questions. Answers are strictly derived from the uploaded documents.
The system **finds answers only from the provided documents** — it does **not hallucinate**.

The system combines:
- Document ingestion and cleaning
- Semantic chunking
- Automatic question generation (Qwen via OpenRouter)
- Fine-tuning a DistilBERT QA model
- Semantic search using FAISS
- A FastAPI-based chat API

What makes this project special?

- It trains its own AI model (DistilBERT)
- It understands document meaning, not just keywords.
- It combines:
  - Semantic Search
  - Fine-Tuned QA models
  - Rule based logic
  - LLM safety checks (QWEN)
- It is incremental: add new documents anytime.



Repository URL:

https://github.com/sathwikvintha/ChatBot-LLM-Distilbert

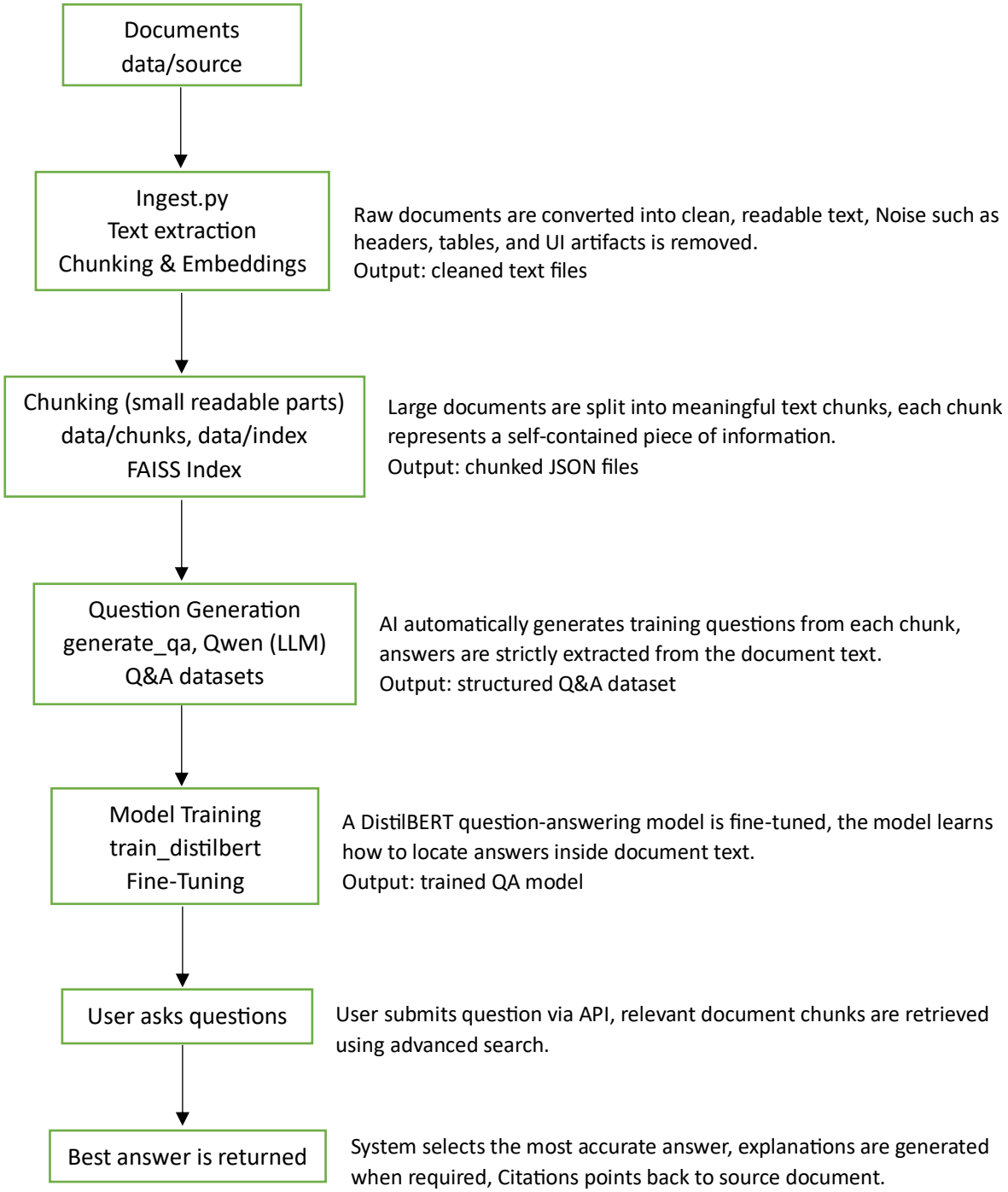
Clone Instructions:

git clone https://github.com/sathwikvintha/ChatBot-LLM-Distilbert.git

cd ChatBot-LLM-Distilbert

2. High-Level Flow (Non-Technical View)

Think of project as a pipeline



3. Project Directory Structure

```
CHATBOT_DISTILBERT/
│
├── app.py                    # API server (FastAPI)
├── ingest.py                 # Document ingestion + chunking
├── generate_qa.py            # Auto-generate questions
├── train_distilbert.py       # Train QA model
├── generate.py               # Answer generation logic
├── qwen2_handler.py          # Qwen safety & grounding logic
│
├── data/
│   ├── source/               # 📥 Raw documents (input)
│   ├── processed/            # Cleaned text
│   ├── chunks/               # Chunked text (JSONL)
│   ├── embeddings/           # Vector embeddings (.npy)
│   ├── index/                # FAISS index + metadata
│
├── domain_data_flat.json     # Training Q&A dataset
├── distilbert-finetuned/     # Trained DistilBERT model
│
├── requirements.txt          # Python dependencies
├── README.md                 # Project documentation
└── .venv/                    # Python virtual environment
```

Path/File	Created By	When it is Created	Should be Edited Manually?	Purpose
data/source	Human	Before running ingest.py	✅ Yes	Raw documents (PDF, DOCX, TXT) provided by users.
data/processed/ data/chunks/ data/embeddings/ data/index/	Script (ingest.py)	During Ingestion	❌ No	Cleaned and normalized text extracted from documents. Small text chunks used for embeddings and QA. Vector embeddings (.npy) for each document. FAISS index and metadata for fast retrieval.
domain_data_flat.json	Script (generate_qa.py)	During QA generation	❌ No	Training QA dataset (extractive only)
distilbert-finetuned/	Script (train_distilbert.py)	After training	❌ No	Fine-tuned DistilBert model
.venv/	Human / Tooling	Environment Setup	❌ No	Python virtual environment
Ingest.py	Human	Initial Setup	✅ Yes (Don't change the flow)	Ingestion pipeline logic
generate_qa.py	Human	Initial Setup	✅ Yes (Don't change the flow)	Auto-generates QA training data
train_distilbert.py	Human	Initial Setup	✅ Yes (Don't change the flow)	Trains extractive QA model
generate.py	Human	Initial Setup	✅ Yes (Don't change the flow)	Runtime answer routing logic
app.py	Human	Initial Setup	✅ Yes (Don't change the flow)	FastAPI entry point
qwen2_handler.py	Human	Initial Setup	✅ Yes (Don't change the flow)	Local Qwen relevance + fallback logic
requirements.txt	Human	Initial Setup	✅ Yes	Dependency List
README.md	Human	Anytime	✅ Yes	Project Documentation

4. Detailed Script Explanation

4.1 ingest.py – Document Ingestion Engine

Purpose

Converts raw documents into **clean, searchable chunks** and builds a **FAISS vector index**.

What it does step by step:

1) Reads documents from: data/source.
2) Supports: PDF, DOCX, TXT / MD
3) Cleans: UI noise, tables, separators, OCR junk (unreadable or nonsensical strings of characters)
4) Normalized text
5) Splits text into semantic chunks
6) Generates embeddings using all-MiniLM-L6-v2 (It is a popular, efficient, deep learning model from Sentence Transformers library that converts sentences and paragraphs into dense vector embeddings, ideal for semantic search, clustering, and information retrieval)
7) Stores:
    a. chunks -> data/chunks/*.jsonl
    b. embeddings -> data/embeddings /*.npy
    c. FAISS index -> data/index

Output

- Chunked data
- Embeddings
- FAISS index
- Metadata file

Run Command

```
python ingest.py
```

When python ingest.py is executed, the ingestion pipeline processes **all documents** present in data/source/.

Expected usage pattern:

- Add **new documents** into data/source/
- Re-run python ingest.py
- The pipeline rebuilds:
    - Cleaned text
    - Chunks
    - Embeddings
    - FAISS index

Although the script reprocesses existing documents, this is intentional and ensures:

- A fully consistent FAISS index
- No stale embeddings, no mismatch between chunks and index

4.2 generate_qa.py – Automatic Question Generator

Purpose

Automatically generates **training questions** from document chunks using **Qwen (OpenRouter)**.

Why this is needed

DistilBERT needs **question–answer pairs** to learn.
Humans don't write these manually — AI generates them.

What it does:

1) Reads chunks from: data/chunks/
2) For each chunk:
    a. Generates 20 questions
    b. Ensures answers are exact substrings
3) Uses: qwen/qwen-2.5-7b-instruct (It is a powerful, 7-billion parameter, instruction-tuned LLM designed for complex tasks)
4) Skips chunks already processed
5) Shows live progress counter
6) Stores result in: domain_data_flat.json

Smart behaviour

- Safe to re-run
- Incremental
- Deduplicated
- Cost-efficient

Run Command

```
python generate_qa.py
```

this script **does not regenerate questions for all files every time**.

Instead, it follows an **incremental and safe workflow:**

- The script scans all chunk files present under `data/chunks/`
- Each chunk has a unique `chunk_id`
- For every chunk, the script checks whether questions for that `chunk_id` already exist in `domain_data_flat.json`

Decision logic:

- ✅ If questions for a chunk already exist (and meet the required count), that chunk is **skipped**
- ✅ If questions are missing or incomplete for a chunk, **only those questions are generated**
- The script then automatically moves on to the **next chunk**

4.3 train_distilbert.py – Model Training

Purpose

Fine-tunes **DistilBERT** to answer questions from your documents

What it does:

1) Loads Q&A dataset: domain_data_flat.json
2) Normalizes text
3) Aligns answer spans
4) Trains: distilbert-base-uncased (It is smaller, faster, and lighter version of the BERT language model, created using knowledge distillation, making it highly efficient for various NLP tasks like text classification, question answering, and embeddings)
5) Saves trained model to: distilbert-finetuned/

Why DistilBERT?

- Fast
- Lightweight
- Accurate
- Production-ready

Run Command

`python train_distilbert.py` **(** ⚠ Run this **only when new data is added)**

4.4 generate.py – Answer Selection Brain

Purpose

This is the **core intelligence** of the system.
It decides **how to answer a user question** using multiple layers.
Why Multiple Layers? (Accuracy, Safety, Determinism, No hallucination)

Layers used (in order):

1. **Exact match** (Directly returns the stored answer if the user's question exactly matches a previously trained question)
2. **Semantic similarity** (Finds the most semantically similar trained question using embeddings and returns its answer if similarity is high enough)
3. **Keyword-based gating** (Uses keyword overlap to match related questions while preventing incorrect matches for procedural or factual queries)
4. **Step extraction** (Automatically extracts step-by-step instructions from the document when the question asks "how" or "setup" type queries)
5. **Sequence logic** (Determines the final or next step by analyzing sentence order when the question refers to sequence or completion)
6. **Qwen grounded generation** (Uses a large language model to generate a clear, human-readable explanation strictly based on the retrieved document context)
7. **DistilBERT extraction** (As a final fallback, extracts the most relevant answer span directly from the document using a fine-tuned QA model)

4.5 qwen2_handler.py – Safety & Relevance Guard

Purpose

Ensures:

- Questions are relevant
- Answers come **only from documents**

Functions:

- qwen2_check_relevance() (This function **checks whether a user's question is actually related to the given document content**)
- qwen2_answer() (This function **generates the final answer**, but **only using the provided document content**)

Important rule

If answer is not found:

"Not mentioned in the document"

4.6 app.py – API Server

Purpose

Exposes the chatbot via FastAPI

What it does:

1. Loads FAISS index
2. Loads document embeddings
3. Loads trained model
4. Handles /chat requests
5. Returns:
    a. Answer
    b. Citations (source documents)

Run Server

```
uvicorn app:app --reload
```

5. Installation Guide (Step-by-Step)

5.1 Create virtual environment

```
python -m venv .venv
source .venv/bin/activate   # Linux/Mac
.venv\Scripts\activate      # Windows
```

5.2 Install dependencies

```
pip install -r requirements.txt
```

5.3 Set environment variable

```
bash
export OPENROUTER_API_KEY=your_key_here

Windows Powershell
setx OPENROUTER_API_KEY "your_key_here"
```

How to get api key?

- Go to https://openrouter.ai/settings/keys
- Create an account
- Click on Create Api Key
- Give some name and click on create.
- Save the api key generated & paste it in the above code.
- If expired or credit limit reached, create a new one and use it.

6. First-Time Setup (Important)

Run these in order:

```
python ingest.py
python generate_qa.py
python train_distilbert.py
```

This creates:

- Chunks
- Questions
- Trained Model

7. Normal Usage (After Training)

Start UI

```
cd org-docs-chatbot
ng serve
```

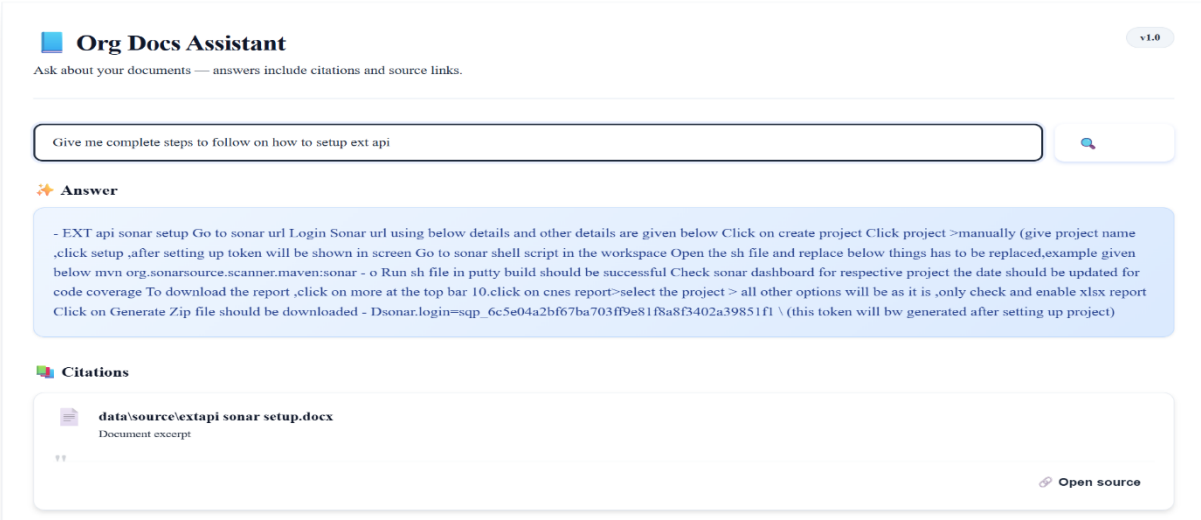
```
(.venv) PS C:\Users\Sathwik Vintha\Desktop\chatbot_distilbert\org-docs-chatbot> ng serve
Initial chunk files | Names         | Raw size
main.js             | main          | 41.55 kB |
styles.css          | styles        | 96 bytes |

                    | Initial total | 41.65 kB

Application bundle generation complete. [4.641 seconds] - 2026-01-15T17:48:05.831Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

Start API

```
uvicorn app:app -reload -port 8000    (on another terminal)
```

```
(.venv) PS C:\Users\Sathwik Vintha\Desktop\chatbot_distilbert> uvicorn app:app --reload --port 8000
INFO:     Will watch for changes in these directories: ['C:\\Users\\Sathwik Vintha\\Desktop\\chatbot_distilbert']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [35640] using StatReload
2026-01-15 23:18:28,732 [INFO] Loading FAISS index and metadata...
2026-01-15 23:18:28,750 [INFO] Use pytorch device_name: cpu
2026-01-15 23:18:28,750 [INFO] Load pretrained SentenceTransformer: sentence-transformers/all-MiniLM-L6-v2
2026-01-15 23:18:33,640 [INFO] FAISS loaded in 4.91s
2026-01-15 23:18:33,642 [INFO] Precomputing document embeddings...
Batches: 100%|                                                    | 1/1 [00:00<00:00,  3.56it/s]
Batches: 100%|                                                    | 1/1 [00:00<00:00,  2.96it/s]
Batches: 100%|                                                    | 1/1 [00:00<00:00,  4.17it/s]
2026-01-15 23:18:34,606 [INFO] Precomputed embeddings for 3 documents
2026-01-15 23:18:34,608 [INFO] Loading training QA data...
Batches: 100%|                                                    | 15/15 [00:03<00:00,  3.86it/s]
INFO:     Started server process [26300]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Result



```
2026-01-15 23:21:25,329 [INFO] Received /chat request
Batches: 100%|                                                    | 1/1 [00:00<00:00, 16.29it/s]
2026-01-15 23:21:25,413 [INFO] Retrieved 8 chunks in 0.08s
Batches: 100%|                                                    | 1/1 [00:00<00:00, 17.26it/s]
Batches: 100%|                                                    | 1/1 [00:00<00:00, 18.60it/s]
2026-01-15 23:21:28,388 [INFO] ---- Layer Progression ----
2026-01-15 23:21:28,388 [INFO] Layer 1: Exact Match ❌
2026-01-15 23:21:28,388 [INFO] Layer 2: Semantic Match ❌
2026-01-15 23:21:28,390 [INFO] Layer 3: Keyword Match (gated) ❌
2026-01-15 23:21:28,390 [INFO] Layer 4: Steps Fallback ✅
2026-01-15 23:21:28,390 [INFO] Layer 5: Sequence ❌
2026-01-15 23:21:28,390 [INFO] Layer 6: Qwen ❌
2026-01-15 23:21:28,390 [INFO] Layer 7: DistilBERT ❌
2026-01-15 23:21:28,390 [INFO] --------------------------
2026-01-15 23:21:28,391 [INFO] /chat processed in 3.06s
INFO:     127.0.0.1:56357 - "POST /chat HTTP/1.1" 200 OK
```

This is on console (vs-code) so it basically tells us on which search layer it got the most accurate answer from and also gives us time consumed for that respective search.

8. Adding New Documents Later

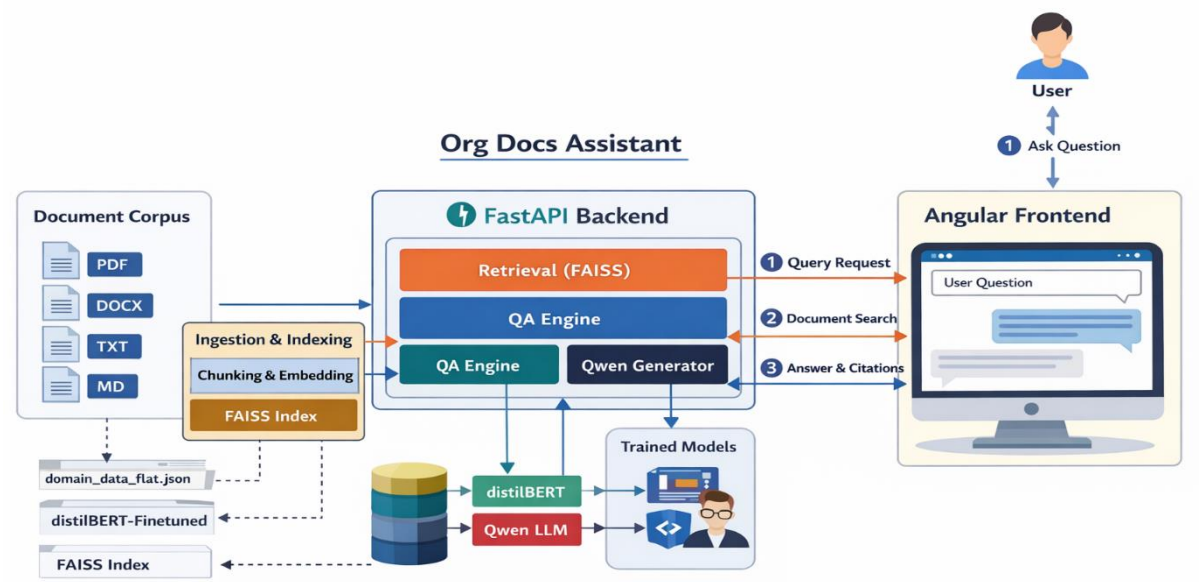
1. Add files to:

```
data/source/
```

2. Run:

```
python ingest.py
python generate_qa.py
python train_distilbert.py
```

only **new documents** are processed.



9. Complete Detailed Execution Flow

```
From UI Keystroke -> Backend -> Models -> Answers -> UI Render
```

9.1 Frontend (Angular UI) – MICRO-LEVEL Flow

1) Browser loads application

File: `index.html`

```
<app-root></app-root>
```

- Browser loads angular bundle (one or more combined JavaScript files containing all the application's compiled code, dependencies (libraries), HTML, CSS, and other assets that are sent to the client's browser).
- <app-root> is the bootstrap anchor tag.

2) Angular bootstraps app

File: `main.ts`

```
bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(),
    importProvidersFrom(FormsModule),
    provideRouter(routes)
  ],
});
```

- Angular starts the application.
- Registers: HTTP client (for backend calls), Forms, Router.

3) Root component loads

File: `app.ts`

```
template: `
  <main class="app-main">
    <router-outlet></router-outlet>
  </main>
```

- Angular prepares routing container.
- No UI is involved here yet, waiting for routes.

4) Routing resolves Chat page

File: `app.routes.ts`

```
{ path: '', component: ChatComponent }
```

- Default route `(/)`.
- Angular instantiates `ChatComponent`.

9.2 Chat UI Rendering (Static)

5) Chat Component initializes

File: `chat/chat.ts`

```
question = '';
answer = '';
citations = [];
loading = false;
errorMessage = '';
```

- Initial UI state.
- No backend call yet.

6) Chat HTML rendered

File: `chat/chat.html`

```
<input [(ngModel)]="question">
<button (ngSubmit)="askQuestion()">
```

- `ngModel` (It is a core feature used to achieve **two-way data binding** between an HTML form element (<input>, <select> & <textarea>) and a property in the component's TypeScript class.
- Button click triggers `askQuestion()`.

9.3 User Action -> Angular Logic

7) User types the question

- Each keystroke updates:

```
this.question (no backend call yet).
```

8) User clicks Search

File: `chat/chat.ts`

```
askQuestion() {
  const q = this.question?.trim();
```

- Input sanitized.
- Empty input blocked.

9) UI state reset

```
this.loading = true;
this.answer = '';
this.citations = [];
this.errorMessage = '';
```

- Spinner shown.
- Empty input blocked.

10) HTTP request sent to backend

```
this.http.post('http://127.0.0.1:8000/chat', {
  question: q,
  top_k: 8
})
```

- Browser sends POST request.
- Control leaves frontend, now UI thread is free.

9.4 FASTAPI Backend Entry

11) FastAPI receives request

File: `app.py`

```
@app.post("/chat")
def chat(q: Query):
```

- FastAPI parses JSON.
- Maps to the model (question: str).

12) Logging starts

```
logging.info("Received /chat request")
```

- Request logged.
- Useful for tracing.

9.5 Retrieval Pipeline (FAISS)

13) `retrieve()` called

```
chunks = retrieve(q.question, q.top_k)
```

14) Question embedding created

```
q_emb = embed_model.encode([question])
```

- Converts text → numeric vector.
- Uses `all-MiniLM-L6-v2`.

15) FAISS search

```
scores, idxs = index.search(q_emb, k)
```

- FAISS finds nearest vectors.
- Returns chunk indexes.

16) Chunk text reloaded

```
meta["text"] = get_chunk_text(meta["chunk_id"])
```

- Reads actual text from `data/chunks/*.jsonl`
- Avoids storing text inside FAISS.

17) Retrieval result returned

```
return results
```

- List of relevant chunks passed forward.

9.6 Answer Generation Core (generate.py)

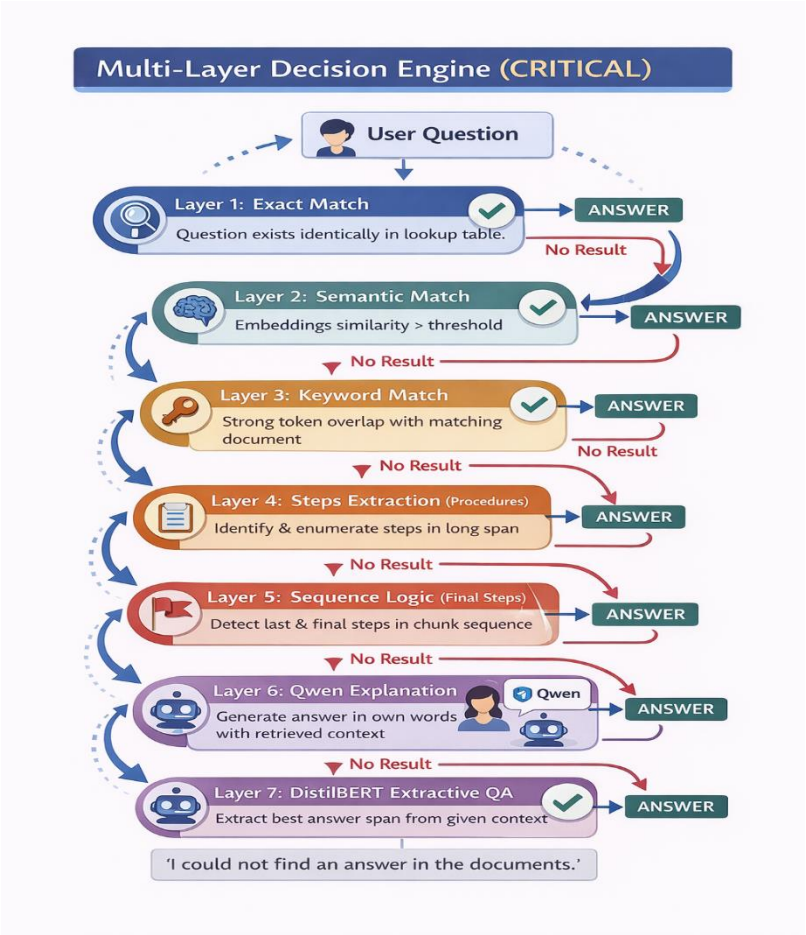
18) `generate_answer()` involved

```
answer, citations = generate_answer(
  question, chunks, DOC_EMBEDDINGS
)

This is brain of the system
```

19) Document-level focusing

```
select_best_document_chunks()
```

- Prevents mixing documents.
- Chooses best document via embeddings.



20) Citations prepared

```
citations = dedupe_chunks(focused_chunks)
```

- Removes duplicate sources.
- Prepares UI-friendly citation.

21) Final response returned

```
return answer, citations
```

9.7 FASTAPI Response

22) JSON response sent

```
{
  "answer": "...",
  "citations": [...]
}
```

- HTTP response sent to browser.

9.8 UI Receives Response

23) Angular receives response

```
next: (resp) => {
  this.answer = resp.answer;
  this.citations = resp.citations;
}
```

- Answer stored.
- Citations stored.

24) UI updates triggered

```
this.cdr.detectChanges();
```

- Forces UI refresh.
- Especially important for SSR.

25) Spinner removed

```
this.loading = false;
```

26) Answer displayed & Citations rendered

File: `chat.html`

```
<p class="answer-text">{{ answer }}</p>
```

- User can now read answer, click source, verify info.